

TDDE62-Lab3

Ahmad Soltani & Simon Boller

March 2026

1 Introduction

The main objective of this lab is to implement a login token for an e-banking system using a simulated Trusted Platform Module (TPM). The implementation is performed inside an Apptainer environment through the command line using the `tpm2-tools` package.

This lab consists of two main assignments. The first focuses on securely importing a shared secret into the TPM to prevent device cloning, while the second assignment ensures code integrity by binding the key to a specific software state using PCR measurements.

1.1 Assignment 1

1.1.1 Design and Implementation (Task 2.1.2 & 2.1.3)

To securely install the shared secret without exposing it to attackers, we designed a process that leverages the TPM's owner hierarchy.

First, the `NVChip` file was deleted to reset the simulated TPM state. All intermediate files generated during the setup were directed to a temporary `scratch` directory to keep the main environment clean and prevent secret leakage.

The initialization procedure consists of the following steps:

- We converted the raw hex shared secret from the bank into a binary file using the provided `hex_to_bin` utility.
- We generated a primary key using `tpm2_createprimary -c scratch/primary.ctx`. This acts as the root encryption key in the TPM's owner hierarchy.
- We packaged the secret key using `tpm2_import -C scratch/primary.ctx -G hmac:sha256 -i scratch/secret.dat -u scratch/hmac.pub -r scratch/hmac.priv`. This wraps the raw `secret.dat` into an HMAC-SHA256 key protected by the primary key, outputting encrypted public and private files.
- We loaded the wrapped key into the TPM's volatile memory using `tpm2_load`, which outputs an active memory handle (`hmac.ctx`).

- Finally, to ensure the key survives power cycles, we moved it to the TPM's Non-Volatile RAM (NVRAM) at the standard user handle 0x81000000 using `tpm2_evictcontrol`.

1.1.2 Anti Cloning Defense

Device cloning occurs when an attacker with physical access copies the token's filesystem to unauthorized hardware. Our design prevents this because the plaintext shared secret never exists on the persistent storage during normal operation.

During initialization, the secret is immediately encrypted by the TPM's unique primary key into the `hmac.priv` file. Even if an attacker copies the entire filesystem, the encrypted files are useless on another machine because the attacker's physical TPM lacks the original primary key required to decrypt them. Furthermore, the temporary `scratch` directory is automatically deleted after execution.

1.1.3 Testing and Verification (Task 2.1.4)

After granting our user write permissions to the local `init.sh` and `response.sh` files, we automated the commands. To prevent directory errors, `mkdir -p scratch` was added to the scripts to generate the temporary workspace at runtime.

When a challenge is received from the server, the `response.sh` script writes it to a temporary file and executes `tpm2_hmac -c 0x81000000`. The TPM calculates the math internally and outputs a raw binary hash. We formatted this output using the `binary_to_decimal` function to successfully log into the bank.

init.sh:

```
#!/bin/bash

# Imports. (Do not remove.)
source /usr/local/etc/bash.aliases
source util.sh

# temporary workspace
mkdir -p scratch

# Store shared secret key to file for later use by tpm2... tools
hex_to_bin $1 scratch/secret.dat

# TPM commands to secure the key
tpm2_createprimary -Q -c scratch/primary.ctx
tpm2_import -Q -C scratch/primary.ctx -G hmac:sha256 -i scratch/secret.dat \
    -u scratch/hmac.pub -r scratch/hmac.priv
tpm2_load -Q -C scratch/primary.ctx -u scratch/hmac.pub -r scratch/hmac.priv \
```

```

-c scratch/hmac.ctx
tpm2_evictcontrol -Q -C o -c scratch/hmac.ctx 0x81000000

response.sh:

#!/bin/bash

# Imports. (Do not remove.)
source /usr/local/etc/bash.aliases
source util.sh

# Build the temporary workspace
mkdir -p scratch

# Write the challenge string to a temporary file
echo -n $1 > scratch/challenge.txt

# Compute the HMAC internally using the persistent key
tpm2_hmac -Q -c 0x81000000 scratch/challenge.txt -o scratch/hmac.dat

# Format the final response
binary_to_decimal scratch/hmac.dat

```

1.2 Assignment 2: Ensuring Code Integrity

1.2.1 Design and Implementation (Task 2.2.2 & 2.2.3)

The objective of Assignment 2 is to ensure that the token's usage script (`response.sh`) has not been maliciously modified (e.g., by malware attempting to intercept challenges). To achieve this, we applied the TPM's Platform Configuration Registers (PCRs) and Authorization Policies.

To bind our secret key to this unmodified state, the initialization script (`init.sh`) was upgraded:

- We opened a temporary TPM session using `tpm2_startauthsession`.
- We read the state of PCR 15 using `tpm2_policypcr -l sha256:15 -L scratch/policy.dat`, saving the expected hash as a strict authorization rule.
- During the `tpm2_import` step, we appended the `-L scratch/policy.dat` flag. This permanently integrated the PCR 15 policy to the HMAC key, instructing the TPM to deny access unless the calling software matches this exact hash.

Therefore, the script (`response.sh`) also required modification to satisfy this policy during runtime:

- Before computing the HMAC, we opened a policy session (`--policy-session`).

- We instructed the TPM to measure the current state of PCR 15 into the active session.
- We executed `tpm2_hmac`, passing the proven session via the `-p session:scratch/session.ctx` flag to authorize the cryptographic operation.

1.2.2 Verification and Destruction Test (Task 2.2.4)

To verify the integrity check, we performed a destruction test that simulates a malware infection.

First, the token was initialized with the `response.sh` script, locking the key to its original hash. We successfully requested a challenge and generated a valid 8-digit response.

Next, we simulated an attack by modifying the `response.sh` file (adding a comment line) without reinitializing the token. When we tried to compute a response for a new challenge, the lab infrastructure hashed the altered script and extended the new value to PCR 15.

Because the current PCR 15 value no longer matched the authorization policy permanently bound to the HMAC key, the TPM successfully detected the aberration. This shows that the code integrity requirement is fully satisfied.

init.sh(updated):

```
#!/bin/bash

# Imports. (Do not remove.)
source /usr/local/etc/bash.aliases
source util.sh

# temporary workspace
mkdir -p scratch

# Store shared secret key to file for later use by tpm2... tools
hex_to_bin $1 scratch/secret.dat

#open a session
tpm2_startauthsession -Q -S scratch/session.ctx
tpm2_policyPCR -Q -S scratch/session.ctx -l sha256:15 -L scratch/policy.dat
tpm2_flushcontext -Q scratch/session.ctx

# TPM commands to secure the key
tpm2_createprimary -Q -c scratch/primary.ctx
##tpm2_import -Q -C scratch/primary.ctx -G hmac:sha256 -i scratch/secret.dat \
    ##-u scratch/hmac.pub -r scratch/hmac.priv
tpm2_import -Q -C scratch/primary.ctx -G hmac:sha256 -i scratch/secret.dat
-u scratch/hmac.pub -r scratch/hmac.priv -L scratch/policy.dat

tpm2_load -Q -C scratch/primary.ctx -u scratch/hmac.pub -r scratch/hmac.priv \
```

```
-c scratch/hmac.ctx  
tpm2_evictcontrol -Q -C o -c scratch/hmac.ctx 0x81000000
```

response.sh(updated):

```
#!/bin/bash
```

```
# Imports. (Do not remove.)  
source /usr/local/etc/bash.aliases  
source util.sh
```

```
# Build the temporary workspace  
mkdir -p scratch
```

```
# Write the challenge string to a temporary file  
echo -n $1 > scratch/challenge.txt
```

```
#Open a policy session  
tpm2_startauthsession -Q --policy-session -S scratch/session.ctx  
tpm2_policyPCR -Q -S scratch/session.ctx -l sha256:15
```

```
# Compute the HMAC internally using the persistent key  
##tpm2_hmac -Q -c 0x81000000 scratch/challenge.txt -o scratch/hmac.dat  
tpm2_hmac -Q -c 0x81000000 -p session:scratch/session.ctx scratch/challenge.dat -o scratch/h  
tpm2_flushcontext -Q scratch/session.ctx  
# Format the final response  
binary_to_decimal scratch/hmac.dat
```